

BananaVision

What it is, what it does, and what happens behind the scenes

What BananaVision is

BananaVision is a phone app that answers one question from a single photo: *is this banana still good, and for how long?* People throw away a lot of fruit because they can't tell how much life it has left. A spotty banana might have three good days in it, or it might be mush tomorrow.

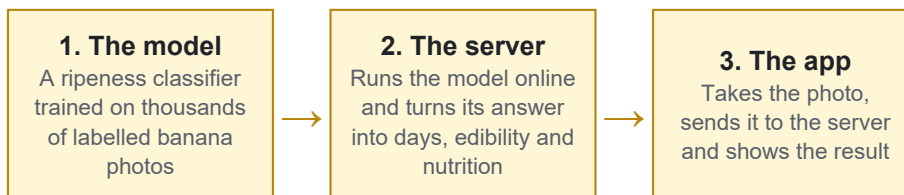
BananaVision looks at the photo and gives a clear answer.

What you get from one photo

Answer	What it means
Ripeness stage	Where the banana is in its life: unripe , ripe , overripe or rotten .
Days left	An estimate of how many days it has before it goes bad, with a low-to-high range.
Edible or not	A plain yes or no. Anything judged rotten is a no.
Nutrition at this stage	What you get from eating it now, what you miss compared with other stages, how much gut-friendly (prebiotic) fibre it has, who it suits and who should go easy, all with cited sources.

The three parts of the project

BananaVision is built from three pieces that work together. Only the first one involves machine learning; the other two are ordinary code.



- **The model** was trained once, ahead of time, in a Python notebook. It only does one thing: look at the pixels and say which of the four stages the banana is in.
- **The server** is a small web service hosted online (on Render). Because the model lives there rather than inside the app, it can be improved without anyone updating the app on their phone.
- **The app** is built with Expo and React Native, so the same code runs on iPhone and Android.

A scan, step by step

Here is everything that happens from opening the app to reading the result. Each step says what you see, then what is going on underneath.

1 Open the app

A welcome screen appears. The first time, the app asks for your name so it can greet you.

Behind the scenes: Your name is saved only on your phone. At the same moment the app quietly pings the server's /health address. The free hosting plan puts the server to sleep after about 15 idle minutes and it takes up to a minute to wake, so this gives it a head start while you're still on the welcome screens.

2 Take or choose a photo

You point the camera at the banana and take a picture, or pick one from your photo library. You get a chance to review it before sending.

Behind the scenes: The app asks for camera permission the first time. If you've turned it off, it explains why it needs it and points you to Settings.

3 The phone prepares the photo

The screen switches to an “analyzing” view.

Behind the scenes: The app first checks that you're online. Then it shrinks the photo so its longest side is 1024 pixels and saves it as a JPEG. A 12-megapixel photo goes from several megabytes to roughly 150 KB, which makes the upload fast. The model only looks at a 224 × 224 image anyway, so nothing useful is lost.

4 The photo is sent to the server

If it's taking a while, the app tells you the server may be waking up, rather than leaving you guessing.

Behind the scenes: The photo is uploaded to the server's /analyze address. The app shows the slow message after 4 seconds and gives up after 90 seconds. You can cancel at any time.

5 The server checks the file

Nothing visible.

Behind the scenes: The server rejects anything over 8 MB and anything that isn't a real image. Phones often record which way up a photo was taken in hidden data rather than rotating the pixels, so the server applies that rotation to make sure the banana is the right way up.

6 Is this actually a banana?

If it isn't, you're told the app couldn't find a banana in the photo.

Behind the scenes: The ripeness model only knows four answers, so it would happily call a patterned wallpaper “unripe”. To stop that, a second, general-purpose image model (MobileNetV2, trained to recognise 1,000 everyday things) looks first. If “banana” isn't among its top 50 guesses, the photo is turned away. That cut-off was chosen by testing: every banana in the test photos passed, and the closest non-banana tried landed at position 106.

7 The ripeness model makes its call

Nothing visible yet.

Behind the scenes: The photo is resized to 224×224 pixels and run through the ripeness model. It doesn't give one answer; it gives four percentages that add up to 100%, one per stage, for example *70% ripe, 25% overripe, 5% unripe, 0% rotten*. The highest one becomes the stage, and its percentage becomes the confidence.

8 Working out the days left

Later you'll see a number of days and a range.

Behind the scenes: Each stage has a rule-of-thumb number of days left: unripe 7, ripe 3, overripe 1, rotten 0. The server blends these using the model's percentages. See the worked example on the next page.

9 Edible or not

A clear yes or no.

Behind the scenes: This is deliberately a simple fixed rule, not a model guess: if the stage is rotten, the answer is no; otherwise yes. Safety decisions are kept as plain code so they're predictable.

10 Adding the nutrition and advice

Nothing visible yet.

Behind the scenes: The server attaches the write-up for that stage from `banana_stages.md`: what's in it, why it's good now, what you miss, risks, who it suits and who should go easy. It also adds the prebiotic fibre figures and the USDA baseline numbers from `banana_nutrition.json`, plus the full list of sources behind every claim.

11 You see the result

The result screen shows the stage on a ripeness scale from green to black, a one-line summary, the days left, whether to eat it, a nutrition label, a gut-health panel and the advice sections. Every figure links to its source.

Behind the scenes: If the model was less than 60% sure, the app flags the result as uncertain. The fixed reference content (sources, nutrition baseline) is also bundled inside the app, so the screen still fills in correctly even if the server is running an older version.

12 If something goes wrong

You get a specific message and a way to try again.

Behind the scenes: The app tells apart: no internet, server unreachable, timed out, not a banana, not a valid image, image too large, a server error, and you cancelling. Each gets its own message instead of a generic “something went wrong”.

Worked example: how “days left” is calculated

A single photo can show what stage a banana is in, but it can't directly show how many days it has left, because that also depends on things the camera can't see: how warm your kitchen is, whether it's in a bag, how it was handled. So BananaVision doesn't try to predict days directly. It predicts the stage, which a photo *can* show, and works the days out from there.

Say the model returns these percentages:

Stage	Model's percentage	Rule-of-thumb days	Contribution
Unripe	5%	7	$0.05 \times 7 = 0.35$
Ripe	70%	3	$0.70 \times 3 = 2.10$
Overripe	25%	1	$0.25 \times 1 = 0.25$
Rotten	0%	0	0
Estimate			about 2.7 days

For the range, the server looks only at stages the model gave at least 10%. Here that's ripe (3 days) and overripe (1 day), so the range is **1 to 3 days**. Unripe, at 5%, is left out.

This gives a smooth answer that behaves sensibly: a banana the model is sure is rotten gets about 0 days, one it's sure is green gets about a week, and one that's between two stages lands in between.

How the model was made

The ripeness model was trained once in the notebook `train/BananaVisionTraining.ipynb`, ideally on a computer with a graphics card (or Google Colab). Here's what that involved:

1. Get labelled photos

A public Kaggle dataset of banana photos, already sorted into folders by ripeness and split into training, validation and test sets. Folder names like “green” or “spotted” are mapped onto the four stages.

2. Start from a model that can already see

Rather than learning to see from scratch, the project starts from MobileNetV2, a small, fast image model already trained on over a million everyday photos. This is called *transfer learning*. Its general knowledge of shapes, colours and textures is kept, and a new final layer is added that outputs the four banana stages.

3. Vary the photos while training

Each training photo is randomly flipped, rotated, zoomed and given different contrast, so the model learns what ripeness looks like rather than memorising particular photos.

4. Train in two rounds

First only the new final layer is trained, for up to 20 rounds. Then the last 30 layers of MobileNetV2 are unlocked and trained gently, at a learning rate 100 times smaller, for up to 10 more. Training stops early once results on the validation photos stop improving, and the best version is kept.

5. Check how well it does

The model is tested on photos it never saw during training, with a breakdown of which stages it confuses with which.

6. Shrink and export it

The model is compressed into a small TensorFlow Lite file (`banana_model.tflite`) that loads quickly on a modest server, along with a settings file (`banana_config.json`) that records the stage order, image size and the days-per-stage numbers.

Where the facts come from

The nutrition and health advice is not produced by the model. It's written by hand in `api/banana_stages.md`, where every claim carries a numbered citation to a published source. The server reads that file by its headings, so it can be edited freely as long as the stage and section names stay the same. The notebook also checks that every stage has all six sections, so a missing one is caught before it reaches users.

The numbers shown as figures (fibre, starch and sugar per stage, and the USDA baseline) live separately in `api/banana_nutrition.json`. A small script, `sync_app_content.py`, copies this reference content into the app so the two never drift apart.

Honest limits

- **Days left is an estimate, not a measurement.** The per-stage day counts are informed rules of thumb. Making it a truly learned prediction would need time-lapse data: the same bananas photographed every day until they rot.
- **Four stages only.** The model doesn't know banana variety, temperature or storage, all of which affect how fast a banana ripens.
- **Needs an internet connection.** The model runs on the server, not on the phone. A fully offline version is a possible future step.
- **Not medical advice.** The health notes are general education. Anyone with a condition, allergy or special diet should follow their own doctor or dietitian.

Project map

Where	What's there
train/	The training notebook, plus the exported model and settings file.
api/app.py	The server: banana check, ripeness model, days and edibility, and the three web addresses (/analyze, /sources, /health).
api/banana_stages.md	The cited, hand-written advice for each stage, and the list of sources.
api/banana_nutrition.json	Nutrition figures per stage and the USDA baseline.
api/imagenet_gate.tflite	The general image model used to check the photo is a banana.
api/Dockerfile	Packages the server so it can be hosted online.
app/	The phone app: screens, components, the code that talks to the server, and bundled reference content.
DaysToBananaDeath_BuildManual.md	A full step-by-step build guide for the whole project.